

C++ & Qt 최종 프로젝트 보고서

ROS2와 Qt를 이용한 Knife Hit 게임 구현

학번: 2026406076

이름: 정창규

제출일: 2026-09-14

목차

1. 프로젝트 개요

1.1 프로젝트 주제 및 선정 이유

1.2 주요 기능

2. 프로그램 구성

2.1 전체 파일 구조

2.2 ROS2 노드 및 통신 구조

2.3 Qt GUI 구성

3. 주요 기능 구현

3.1 원판 회전 및 칼 발사

3.2 칼 충돌 판정

3.3 사과 및 랜덤 장애물 구현

3.4 스테이지 및 난이도 구현

3.5 Game Over, Restart 및 Win 처리

4. 실행 결과

4.1 기본 게임 실행

4.2 스테이지 진행 및 난이도 변화

4.3 충돌, Restart 및 최종 클리어

5. 구현 과정에서 발생한 문제와 해결

6. 느낀 점

1. 프로젝트 개요

1.1 프로젝트 주제 및 선정 이유

이번 프로젝트에서는 C++, Qt, ROS2를 이용하여 Knife Hit 형태의 게임을 구현하였다. 사용자가 Space 키를 누르면 칼이 회전하는 원판을 향해 이동하고, 기존 칼과 충돌하지 않으면 원판에 꽂히도록 구성하였다.

주제는 단순한 GUI 프로그램보다 직접 조작하면서 결과를 확인할 수 있는 프로그램을 만들어보고 싶어서 선택하였다. Knife Hit는 조작은 단순하지만 원판 회전, 충돌 판정, 점수와 스테이지 등 여러 기능을 구현할 수 있어 C++과 Qt, ROS2를 함께 적용하기 좋다고 생각하였다.

1.2 주요 기능

게임은 Stage 1부터 Stage 5까지 진행되도록 구현하였다. 스테이지가 올라갈수록 던져야 하는 칼의 개수와 원판의 회전 속도가 증가하고, 일부 스테이지에서는 회전 방향도 반대로 설정하였다.

원판에는 장애물 칼과 사과가 생성되며, 위치는 매번 랜덤하게 정해지도록 하였다. 칼이 기존 칼과 충돌하면 GAME OVER가 되고, 사과를 맞으면 획득 개수가 증가한다.

현재 스테이지의 칼을 모두 사용하면 STAGE CLEAR가 표시되고 다음 스테이지로 넘어간다. Stage 5까지 완료하면 YOU WIN 화면을 표시하며, Restart 버튼이나 R 키를 이용하여 게임을 처음부터 다시 시작할 수 있도록 하였다.

그리고 ROS2에서는 GUI와 게임 로직을 각각 나누어 구성하였다. GUI에서 칼 발사 신호를 보내면 Logic Node에서 충돌과 점수, 스테이지 상태를 계산하고 그 결과를 다시 GUI에 전달하도록 하였다.

2. 프로그램 구성

2.1 전체 파일 구조

프로젝트는 `ros2_create_qt_pkg`를 이용하여 ROS2와 Qt가 함께 동작할 수 있는 형태로 패키지를 생성한 뒤, 게임에 필요한 파일을 추가하여 구성하였다.

```
knife_hit_game/  
├─ CMakeLists.txt  
├─ package.xml  
├─ include/  
│   └─ knife_hit_game/  
│       ├── main_window.hpp  
│       └─ qnode.hpp  
├─ src/  
│   ├── main.cpp  
│   ├── main_window.cpp  
│   ├── qnode.cpp  
│   └─ game_logic_node.cpp  
├─ ui/  
│   └─ mainwindow.ui  
└─ resources/  
    ├── images.qrc  
    └─ images/
```

`game_logic_node.cpp`에서는 원판 회전, 칼 충돌 판정, 점수, 사과, 스테이지와 같은 게임 로직을 처리하였다.

`qnode.cpp`에서는 ROS2 Topic과 Service 통신을 담당하며, QThread에서 ROS2 spin이 실행되도록 구성하였다. `main_window.cpp`에서는 칼 발사 애니메이션과 원판, 사과, 남은 칼 등의 화면 표시를 처리하였다.

`mainwindow.ui`는 Qt Designer로 만든 화면 구성을 저장하고 있으며, 원판과 칼, 사과 이미지는 `resources/images` 폴더에 저장하여 `images.qrc`를 통해 불러오도록 하였다.

2.2 ROS2 노드 및 통신 구조

프로그램은 `game_logic_node`와 `game_gui_node` 두 개의 ROS2 노드로 구성하였다.

`game_logic_node`에서는 원판 회전, 칼 충돌 판정, 점수, 사과와 스테이지 등 실제 게임 상태를 계산하고, GUI 쪽에서는 QNode와 MainWindow를 이용하여 ROS2 통신과 화면 출력을 나누어 처리하였다.

GUI의 QNode는 QThread를 상속받아 별도의 Thread에서 ROS2의 `spin_some()`이 실행되도록 하였다. 이를 통해 Qt의 화면 처리와 ROS2의 Topic 및 Service 통신이 서로 방해하지 않고 동작하도록 구성하였다.

칼이 원판에 도착하면 GUI의 QNode에서 `/knife_throw` Topic을 발행한다.

`game_logic_node`에서는 이를 받아 칼이 꽂힐 각도와 충돌 여부를 계산하고, 점수와 스테이지 등의 게임 상태를 `/game_state` Topic을 통해 다시 GUI로 전달한다. QNode가 받은 데이터는 Qt Signal을 이용하여 MainWindow로 전달하고 화면에 반영하도록 하였다.

게임을 다시 시작할 때에는 `/reset_game` Service를 사용하였다. Restart 버튼이나 R 키를

누르면 GUI에서 Service 요청을 보내고, game_logic_node에서 점수, 사과 개수, 스테이지 등의 상태를 초기화한 뒤 새로운 게임 상태를 다시 GUI에 전달하도록 하였다.

2.3 Qt GUI 구성

Qt 화면은 Qt Designer를 이용하여 구성하였다. 점수, 현재 스테이지, 획득한 사과 개수와 남은 칼 개수는 Label을 이용하여 표시하였고, 실제 게임이 진행되는 부분은 QGraphicsView와 QGraphicsScene을 사용하였다.

원판, 발사되는 칼, 원판에 꽂힌 칼과 사과는 QGraphicsScene에 이미지 형태로 추가하였다. 원판과 함께 움직여야 하는 칼과 사과는 하나의 QGraphicsItemGroup으로 묶고, 그룹 전체를 회전시키는 방식으로 구현하였다.

GUI의 실제 화면 처리는 MainWindow 클래스에서 담당하도록 하였다. MainWindow에서는 칼 발사 애니메이션, 원판 회전, 사과와 칼의 표시, 점수 및 스테이지 UI 갱신을 처리하였다.

또한 QNode에서 전달받은 /game_state 데이터를 MainWindow에서 분석하여 화면에 반영하도록 하였다. 따라서 Qt에서는 충돌 여부나 점수를 직접 계산하지 않고, Logic Node에서 계산된 결과를 받아 화면에 표시하는 구조로 구성하였다.

3. 주요 기능 구현

3.1 원판 회전 및 칼 발사

원판은 game_logic_node에서 현재 스테이지의 속도와 회전 방향을 이용하여 각도를 계속 변경하도록 하였다.

```
disk_angle_ +=
    speedForStage(stage_) *
    rotation_direction_;

disk_angle_ =
    normalizeAngle(disk_angle_);
```

speedForStage()에서는 스테이지별 회전 속도를 가져오고, rotation_direction_ 값이 1이면 정방향, -1이면 역방향으로 회전하도록 하였다. 계산된 각도는 normalizeAngle()을 이용하여 항상 0도에서 360도 사이의 값으로 유지하였다.

Qt 화면에서는 원판의 회전 중심을 잡기 위해 원판, 꽂힌 칼, 사과를 하나의 rotating_group_으로 묶었다. 그룹의 위치는 (280, 210)으로 설정하고 원판 이미지의 중심이 그룹의 (0, 0)에 오도록 배치하였다.

```
rotating_group->setPos(
    280.0,
    210.0);

disk_item->setPos(
    -(rect.width() / 2.0 + 2.5),
    -(rect.height() / 2.0));
```

이후 rotating_group_의 각도를 변경하면 원판뿐만 아니라 원판에 꽂힌 칼과 사과도 같은 중심을 기준으로 함께 회전한다.

```
visual_disk_angle_ +=
    diff * 0.65;

rotating_group->setRotation(
    visual_disk_angle_);
```

사과 역시 원판 중심을 기준으로 위치를 계산하였다. 원판 중심에서 145만큼 떨어진 원 위에 사과를 배치하고, 사과 이미지의 중심이 계산된 위치에 오도록 이미지 크기의 절반을 빼주었다.

```
const double radius =
    145.0;

double x =
    -std::sin(rad) * radius;

double y =
    std::cos(rad) * radius;

item->setPos(
    x - rect.width() / 2.0,
    y - rect.height() / 2.0);
```

사과를 rotating_group_의 자식으로 생성하였기 때문에 원판이 회전하더라도 사과가 원판에 붙어 있는 것처럼 함께 움직이도록 할 수 있었다. 사과가 어떤 각도에 랜덤으로 생성되는지는 뒤의 사과 및 랜덤 장애물 구현에서 설명하였다.

칼 발사는 사용자가 Space 키를 누르면 시작하도록 하였다. throwKnife()가 실행되면 칼이 이미 날아가는 중인지 확인한 뒤 Timer를 시작한다.

```
knife_flying_ = true;
knife_timer->start();
```

Timer가 실행되는 동안 칼의 Y 좌표를 일정하게 감소시켜 화면 아래쪽에서 원판 방향으로 이동하도록 하였다.

```
pos.setY(  
    pos.y() - 14.0);  
  
knife_item->setPos(  
    pos);
```

칼이 원판 위치까지 도착하면 QNode의 publishThrow()를 실행한다.

```
qnode->publishThrow();
```

이때 /knife_throw Topic이 game_logic_node로 전달되고, Logic Node에서 칼이 꽂힐 위치와 기존 칼과의 충돌 여부를 판단한다. 충돌하지 않은 경우 새로운 칼의 위치와 변경된 게임 상태를 /game_state Topic으로 다시 GUI에 전달하여 화면에 반영하도록 하였다.

3.2 칼 충돌 판정

칼이 원판에 도착하면 game_logic_node에서 현재 원판의 각도를 이용하여 새 칼이 꽂힐 위치를 계산한다.

```
double new_angle =  
    normalizeAngle(  
        -disk_angle_);
```

원판이 회전하고 있기 때문에 화면에서 칼이 항상 같은 위치로 올라오더라도, 원판 기준으로 보면 칼이 꽂히는 각도는 계속 달라진다. 그래서 현재 원판의 회전 각도를 반대로 적용하여 원판 기준의 칼 위치를 구하도록 하였다.

그다음 기존에 꽂혀 있는 칼의 각도와 새 칼의 각도를 하나씩 비교하였다.

```
for (double old_angle :  
    knife_angles_)  
{  
    if (angleDifference(  
        new_angle,  
        old_angle) <  
        18.0)  
    {  
        game_over_ = true;  
  
        publishState();  
  
        return;  
    }  
}
```

angleDifference() 함수는 두 각도 사이의 가장 짧은 거리를 계산한다.

```
double angleDifference(  
    double a,  
    double b)  
{  
    double diff =  
        std::fabs(a - b);  
  
    if (diff > 180.0)  
        diff = 360.0 - diff;  
  
    return diff;  
}
```

예를 들어 한 칼이 350도에 있고 새로운 칼이 10도에 꽂히는 경우 단순히 빼면 340도가 나오지만, 실제로는 두 칼 사이의 거리가 20도이다. 따라서 차이가 180도보다 크면 $360 - \text{diff}$ 를 이용하여 실제 가까운 각도 차이를 구하였다.

두 칼의 각도 차이가 18도보다 작으면 서로 충돌한 것으로 판단하여 game_over_를 true로 변경한다. 반대로 충돌하지 않았다면 새 칼의 각도를 knife_angles_에 추가한다.

```
knife_angles_.push_back(  
    new_angle);  
  
score_++;  
  
remaining_knives--;
```

이후 점수를 1 증가시키고 남은 칼의 개수를 감소시킨다. 변경된 칼의 위치와 게임 상태는 /game_state Topic을 통해 GUI로 전달되고, GUI에서는 새로운 칼 이미지를 원판에 추가하여 함께 회전하도록 하였다.

즉 충돌 판정은 칼 이미지 자체가 겹치는지를 직접 확인하는 방식이 아니라, 원판에 저장된 칼들의 각도 차이를 이용하는 방식으로 구현하였다.

3.3 사과 및 랜덤 장애물 구현

사과와 장애물 칼은 스테이지가 시작될 때 랜덤한 각도로 생성되도록 하였다. 단순히 랜덤한 위치를 바로 사용하는 것이 아니라, 이미 생성된 칼이나 사과와 너무 가까운 위치인지 확인한 뒤 배치하도록 하였다.

먼저 0도에서 360도 사이의 랜덤 각도를 생성하였다.

```
double randomAngle()
{
    std::uniform_real_distribution<double> dist(
        0.0,
        359.999);

    return dist(rng_);
}
```

새로운 위치를 정할 때는 기존에 있는 물체들과의 각도 차이를 확인한다.

```
if (angleDifference(
    candidate,
    angle) <
    min_distance)
{
    return false;
}
```

장애물 칼은 서로 너무 가까이 생성되지 않도록 최소 42도의 간격을 두었고, 사과는 기존 칼과 다른 사과의 위치를 모두 확인하여 최소 35도의 간격을 유지하도록 하였다.

```
double angle =
    createSafeRandomAngle(
        knife_angles_,
        42.0);
```

사과를 생성할 때에는 기존 장애물 칼과 이미 생성된 사과의 각도를 하나의 목록으로 만든 뒤 새로운 위치를 결정하였다.

```
std::vector<double> blocked =
    knife_angles_;

blocked.insert(
    blocked.end(),
    apple_angles_.begin(),
    apple_angles_.end());
```

이후 안전한 위치를 찾으면 해당 각도를 apple_angles_에 저장하였다.

```
double angle =
    createSafeRandomAngle(
        blocked,
        35.0);

apple_angles_.push_back(
    angle);
```

사과에 칼이 맞았는지도 각도 차이를 이용하여 판정하였다.

```
if (angleDifference(
    new_angle,
    *it) <
    28.0)
{
    apple_count++;

    apple_angles_.erase(
        it);

    break;
}
```

새로 쏜 칼과 사과의 각도 차이가 28도보다 작으면 사과를 맞힌 것으로 판단하였다. 이 경우 사과 획득 개수를 1 증가시키고, 맞은 사과의 각도를 목록에서 삭제하여 화면에서도 사라지도록 하였다.

즉 사과와 장애물 칼은 0~360도 사이에서 랜덤한 위치를 정하되, 다른 물체와 너무 가까우면 다시 위치를 정하는 방식으로 구현하였다. 이렇게 하여 게임을 다시 시작할 때마다 사과와 장애물의 위치가 달라지도록 하였다.

3.4 스테이지 및 난이도 구현

게임은 총 5개의 스테이지로 구성하였고, 스테이지가 올라갈수록 필요한 칼 개수와 원판 속도, 장애물 개수가 증가하도록 하였다.

먼저 스테이지별로 사용해야 하는 칼의 개수를 다르게 설정하였다.

```
int knivesForStage(int stage)
{
    switch (stage)
    {
        case 1: return 4;
        case 2: return 5;
        case 3: return 6;
        case 4: return 7;
        case 5: return 8;
        default: return 4;
    }
}
```

Stage 1에서는 4개의 칼을 사용하고, 마지막 Stage 5에서는 8개의 칼을 사용하도록 하여 뒤로 갈수록 더 많은 칼을 충돌 없이 꽂아야 한다.

원판의 회전 속도도 스테이지별로 다르게 설정하였다.

```
double speedForStage(int stage)
{
    switch (stage)
    {
        case 1: return 0.40;
        case 2: return 0.65;
        case 3: return 0.90;
        case 4: return 1.20;
        case 5: return 1.55;
        default: return 0.40;
    }
}
```

스테이지가 올라갈수록 한 번의 업데이트에서 변경되는 각도가 커지기 때문에 원판이 더 빠르게 회전한다.

회전 방향에도 차이를 주었다.

```
if (stage == 3 ||
    stage == 5)
{
    rotation_direction_ = -1.0;
}
else
{
    rotation_direction_ = 1.0;
}
```

Stage 1, 2, 4는 정방향으로 회전하고 Stage 3과 Stage 5는 반대 방향으로 회전하도록 하였다. 한 스테이지가 진행되는 도중에는 방향이 바뀌지 않고, 새로운 스테이지가 시작될 때만 방향이 결정된다.

스테이지가 올라가면 처음부터 꽂혀 있는 장애물 칼의 개수도 증가한다.

```
int obstacleKnivesForStage(int stage)
{
    switch (stage)
    {
        case 1: return 0;
        case 2: return 1;
        case 3: return 2;
        case 4: return 3;
        case 5: return 3;
        default: return 0;
    }
}
```

사과의 개수 역시 스테이지에 따라 증가하며 일부 스테이지에서는 범위 안에서 랜덤하게 정해지도록 하였다.

```
case 2:
    return randomInt(1, 2);

case 4:
    return randomInt(2, 3);

case 5:
    return randomInt(3, 4);
```

따라서 스테이지가 올라갈수록 원판 속도 증가, 필요한 칼 개수 증가, 장애물 증가, 사과 개수 변화, 회전 방향 변경이 함께 적용되어 난이도가 높아지도록 구성하였다.

3.5 Game Over, Restart 및 Win 처리

칼이 기존 칼과 충돌하면 `game_over_` 값을 `true`로 변경하고 현재 게임 상태를 GUI로 전달하도록 하였다.

```

if (angleDifference(
    new_angle,
    old_angle) <
    18.0)
{
    game_over_ = true;

    publishState();

    return;
}

```

GUI에서는 /game_state로 전달받은 game_over_ 값을 확인하여 GAME OVER 화면을 표시한다. 이때 점수도 함께 보여주고 Restart 버튼을 나타내도록 하였다.

```

if (game_over_)
{
    showGameOver();
}

```

Restart 버튼이나 R 키를 누르면 QNode를 통해 /reset_game Service를 요청한다.

```

qnode_>resetGame();

```

Logic Node에서는 Service 요청을 받으면 점수, 사과 개수, 스테이지와 게임 상태를 모두 초기화한 뒤 Stage 1을 다시 준비한다.

```

score_ = 0;
apple_count_ = 0;
game_over_ = false;
win_ = false;
stage_ = 1;

prepareStage(1);

```

현재 스테이지에서 남은 칼의 개수가 0이 되면 스테이지가 끝난 것으로 판단한다.

```
if (remaining_knives_ <= 0)
{
    if (stage_ == 5)
    {
        win_ = true;
    }
    else
    {
        stage_clear_ = true;
    }
}
```

Stage 1부터 Stage 4까지는 STAGE CLEAR 화면을 잠시 표시한 뒤 다음 스테이지로 이동하고, 마지막 Stage 5까지 완료한 경우에는 win_ 값을 true로 변경하여 YOU WIN 화면을 표시하도록 하였다.

이렇게 하여 총돌 시 게임 종료, 스테이지 완료 시 다음 단계 이동, 최종 스테이지 완료 시 승리, Restart 시 전체 게임 초기화가 가능하도록 구현하였다.

4. 실행 결과

4.1 기본 게임 실행

프로그램을 실행하면 `game_logic_node`와 `game_gui_node`가 각각 동작하며 Qt 게임 화면이 나타난다. 화면에는 중앙의 회전 원판, 아래쪽의 발사 칼, 현재 스테이지, 점수, 사과 개수와 남은 칼이 표시된다.

Stage 1에서는 장애물 칼 없이 시작하고 사과 1개가 생성된다. 사용자가 Space 키를 누르면 칼이 원판 방향으로 이동하고, 충돌하지 않으면 원판에 꽂힌 뒤 원판과 함께 회전하는 것을 확인하였다.

또한 칼이 정상적으로 꽂힐 때마다 점수가 증가하고 왼쪽의 남은 칼 표시도 하나씩 사용된 상태로 변경되는 것을 확인하였다. 사과 위치에 칼을 맞힌 경우에는 사과가 사라지고 상단의 사과 개수가 증가하였다.

4.2 스테이지 진행 및 난이도 변화

현재 스테이지의 칼을 모두 사용하면 STAGE CLEAR가 표시되고 잠시 후 다음 스테이지로 이동한다. 스테이지가 올라갈수록 던져야 하는 칼의 개수가 증가하고 원판의 회전 속도도 빨라지는 것을 확인하였다.

Stage 3과 Stage 5에서는 원판이 반대 방향으로 회전하도록 하여 이전 스테이지와 다른 조작감을 주었다. 또한 Stage 2부터는 시작할 때 원판에 장애물 칼이 생성되고, 뒤 스테이지로 갈수록 장애물과 사과의 개수도 증가하도록 하였다.

장애물 칼과 사과는 매번 같은 위치에 생성되지 않고 랜덤한 각도에 배치되기 때문에 같은 스테이지를 다시 시작하더라도 원판의 구성이 달라지는 것을 확인할 수 있었다.

4.3 충돌, Restart 및 최종 클리어

새로 발사한 칼이 기존에 꽂혀 있는 칼과 너무 가까운 위치에 들어가면 충돌로 판단되어 GAME OVER 화면이 나타나는 것을 확인하였다. 이 상태에서는 더 이상 칼을 발사할 수 없으며 현재 점수와 Restart 버튼이 함께 표시된다.

Restart 버튼이나 R 키를 누르면 `/reset_game` Service가 실행되어 점수, 획득한 사과 개수, 스테이지와 원판 상태가 모두 초기화되었다. 이후 Stage 1부터 다시 게임을 시작할 수 있었다.

마지막 Stage 5에서 주어진 칼을 모두 충돌 없이 사용하면 YOU WIN 화면이 표시되도록 하였다. 이를 통해 Game Over, Restart, Stage Clear와 최종 클리어까지 전체 게임 흐름이 정상적으로 동작하는 것을 확인하였다.

5. 구현 과정에서 발생한 문제와 해결

프로젝트를 구현하면서 Qt 화면에 사용한 이미지 리소스가 정상적으로 표시되지 않는 경우가 있었다. qrc 파일의 경로나 alias가 맞지 않으면 원판이나 칼 이미지가 나타나지 않았기 때문에, 리소스 경로와 images.qrc 내용을 다시 확인하여 이미지가 정상적으로 불러와지도록 수정하였다.

UI를 구현하는 과정에서는 원판이 회전할 때 화면에서 약간 끊기거나 지지직거리는 것처럼 보이는 문제가 있었다. Logic Node에서 계산한 원판 각도를 GUI에 바로 적용하면 ROS2 통신 주기와 Qt 화면 갱신 주기의 차이 때문에 회전이 부드럽게 보이지 않았다. 이를 해결하기 위해 Logic Node에서 받은 각도를 `target_disk_angle_`에 저장하고, 현재 화면 각도인 `visual_disk_angle_`이 목표 각도를 조금씩 따라가도록 하였다.

```
visual_disk_angle_ +=  
    diff * 0.65;  
  
rotating_group_>setRotation(  
    visual_disk_angle_);
```

이 방식을 적용한 뒤에는 원판이 한 번에 각도가 바뀌는 느낌이 줄어들고 이전보다 부드럽게 회전하였다.

또한 게임을 테스트하는 과정에서 이전에 실행했던 ROS2 노드가 종료되지 않고 남아 있어 원판이 흔들리거나 칼이 여러 번 추가되는 현상이 발생하였다. 같은 Topic을 사용하는 노드가 여러 개 실행되고 있었기 때문에 기존 프로세스를 종료한 뒤 Logic Node와 GUI Node를 각각 하나씩만 실행하여 해결하였다.

처음에는 일반 ROS2 패키지 안에서 Qt를 직접 연결하여 사용하였지만, 프로젝트 조건에 맞추기 위해 `ros2_create_qt_pkg`를 이용한 구조로 다시 구성하였다. 이 과정에서 QNode가 QThread를 상속받아 ROS2 spin을 담당하고, MainWindow에서는 Qt 화면을 처리하도록 역할을 나누었다.

새 구조로 변경한 뒤에는 Q_OBJECT가 포함된 QNode와 MainWindow에서 vtable 관련 링크 오류가 발생하였다. CMakeLists.txt의 GUI 실행 파일에 `qnode.hpp`와 `main_window.hpp`를 빌드 대상으로 추가하여 Qt의 AUTOMOC가 정상적으로 동작하도록 수정하였고 이후 정상적으로 빌드할 수 있었다.

6. 느낀 점

이번 프로젝트를 진행하면서 C++과 Qt, ROS2를 따로 사용하는 것보다 서로 연결해서 하나의 프로그램으로 만드는 과정이 더 어렵다는 것을 느꼈다. 특히 게임 로직과 GUI를 나누고 Topic과 Service를 이용하여 데이터를 주고받는 구조를 직접 구현하면서 ROS2에서 노드를 분리하는 이유를 조금 더 이해할 수 있었다.

처음에는 화면에 원판과 칼을 움직이게 만드는 것 자체에 집중했지만, 이후 충돌 판정이나 랜덤 배치, 스테이지별 난이도까지 추가하면서 단순히 화면만 만드는 것과 실제로 게임이 동작하도록 만드는 것은 많이 다르다는 것을 알게 되었다.

또한 Qt에서 원판이 부드럽게 회전하지 않거나 이미지 리소스가 제대로 나오지 않는 문제, ROS2 노드가 중복 실행되면서 게임이 이상하게 동작하는 문제 등을 직접 해결하면서 코드뿐만 아니라 실행 환경과 구조도 같이 확인해야 한다는 것을 배웠다.

마지막으로 `ros2_create_qt_pkg`를 이용하여 `QNode`, `QThread`, `MainWindow` 구조로 다시 구성하면서 Qt와 ROS2를 연결하는 방법을 더 확실하게 이해할 수 있었다. 처음보다 코드의 각 부분이 어떤 역할을 하는지 구분해서 볼 수 있게 된 점이 가장 도움이 되었다.